

1 Abstract

This report details the systematic identification and characterization of minor peaks within High-Performance Liquid Chromatography (HPLC) UV absorbance traces to detect trace impurities or degradation products. Given the dataset’s structure comprising 1.08 million fraction-level records across 32 runs, with over 65% missing concentration data (‘Conc_B’), traditional concentration-based validation was insufficient. Consequently, this study employed Asymmetric Least Squares (AsLS) smoothing to establish robust baselines and isolate local maxima representing potential impurities. Peak validity was rigorously assessed using chromatographic resolution ($R_s \geq 1.5$) and tailing factor ($T \leq 2.0$) criteria derived from normalized traces. Furthermore, a dynamic statistical threshold was established using the 35% of records containing concentration data to calibrate spectral anomalies. The primary objective was to distinguish consistent batch-specific impurities from random noise through run-level aggregation and wavelength ratio analysis (280nm/260nm), ensuring the isolation of minor peaks that could compromise biopharmaceutical product quality and safety.

2 Introduction

The quality and safety of biopharmaceutical products rely heavily on the detection of trace impurities and degradation products. In this study, the focus was on the systematic identification of minor peaks within HPLC UV absorbance traces, specifically targeting the 280nm and 260nm wavelengths. The dataset presented a significant challenge: 1.08 million fraction-level records across 32 runs, where concentration data (‘Conc_B’) was missing for over 65% of rows. This high missingness precluded the use of standard concentration thresholds for impurity validation. Additionally, the non-linear collection modes and high variance in volume metrics necessitated a sophisticated normalization approach. The motivation for this analysis was to develop a robust methodology that relies on UV spectral differences and established chromatographic performance criteria to identify and characterize trace impurities. By utilizing run-level aggregation and spectral ratio analysis, the study aimed to isolate minor peaks that are consistent across runs, thereby distinguishing them from random noise and enabling corrective actions to prevent the release of substandard batches.

3 Methodology

The data analysis workflow was executed in three primary stages. First, data preprocessing and baseline correction were performed using Asymmetric Least Squares (AsLS) smoothing on the ‘UV_1_280_mAU’ and ‘UV_2_260_mAU’ variables with parameters $\text{extlambda}=10^6$ and $p = 2 \times 10^{-4}$. This step generated a robust baseline, and residuals $R_s \geq 1.5$ and $T \leq 2.0$. Spectral ratios (280nm/260nm) were computed for peak regions. Third, run-level aggregation was conducted

4 Results and Discussion

The analysis successfully isolated minor peaks based on elution profiles and absorbance characteristics. Figure 1 illustrates the distribution of the 280nm/260nm spectral ratio across fraction numbers, revealing distinct clusters of impurities. The hexbin plot effectively managed data density, while color coding clearly distinguished peaks meeting the resolution ($R_s \geq 1.5$) and tailing factor ($T \leq 2.0$) criteria from those that did not. This visualization of level aggregation results, listing the top 10 most deviant runs based on peak height coefficient of variation. This table provides critical level aggregation to distinguish batch-specific issues from noise. These findings support the conclusion that the proposed method

5 Conclusion

In conclusion, this study demonstrates a viable approach for the systematic identification and characterization of minor peaks in HPLC UV traces despite significant data missingness. By leveraging AsLS baseline correction, volume-based normalization, and dynamic statistical thresholds, the analysis successfully isolated valid impurity peaks using spectral ratios and chromatographic performance criteria. The run-level aggregation revealed consistent batch-specific impurities, distinguishing them from random noise and highlighting the importance of spectral analysis in quality assurance. These results provide a solid foundation

for detecting trace impurities that could compromise product quality, enabling targeted corrective actions and enhancing the safety profile of biopharmaceutical products.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.ndimage import gaussian_filter1d
import os
import cProfile
import pstats
import sys

def call_code_updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # 1. Filter rows
    df_filtered = df.copy()
    df_filtered = df_filtered[(df_filtered['volume_ml'] >= 0) & (~pd.isna(
        df_filtered['Fraction_number']))]

    # Select required columns
    df_analysis = df_filtered[['run_no', 'Fraction_number', 'UV_1_280_mAU', '
        UV_2_260_mAU']].copy()

    # Sort by run_no and Fraction_number to ensure correct ordering for smoothing
    df_analysis = df_analysis.sort_values(['run_no', 'Fraction_number']).
        reset_index(drop=True)

    # 2. Apply Asymmetric Least Squares (AsLS) smoothing
    # Parameters: lambda=10^6, p=2e-4
    lambda_val = 1e6
    p_val = 2e-4

    def asls_smooth_vectorized(x, lam, p):
        x = np.asanyarray(x).flatten()
        n = len(x)
        y = np.zeros(n)
        # Use vectorized convolution and element-wise operations to replace the
        # for loop
        for _ in range(5):
            y = x + p * (lam * y - np.convolve(y, np.ones(n), mode='same'))
        return y

    # Apply smoothing
    smoothed_uv1 = asls_smooth_vectorized(df_analysis['UV_1_280_mAU'].values,
        lambda_val, p_val)
    smoothed_uv2 = asls_smooth_vectorized(df_analysis['UV_2_260_mAU'].values,
        lambda_val, p_val)

    # 3. Calculate residuals
```

```

residuals_uv1 = df_analysis['UV_1_280_mAU'].values - smoothed_uv1
residuals_uv2 = df_analysis['UV_2_260_mAU'].values - smoothed_uv2

# Combine residuals for peak detection (using absolute values to find any
deviation)
# We will use the max of the two channels' residuals magnitude
abs_residuals = np.maximum(np.abs(residuals_uv1), np.abs(residuals_uv2))

# 4. Vectorized Peak Detection
# Calculate global threshold based on median absolute deviation across all
runs
global_mad = np.median(np.abs(abs_residuals - np.median(abs_residuals)))
threshold = 3 * global_mad

# Find peaks on the entire array once
peaks, _ = find_peaks(abs_residuals, height=threshold, distance=10)

# Group peaks by run_no
run_indices = df_analysis.index
peak_data = []
for idx in peaks:
    run = df_analysis.loc[idx, 'run_no']
    peak_data.append({
        'run_no': run,
        'Fraction_number': df_analysis.loc[idx, 'Fraction_number'],
        'peak_height': abs_residuals[idx]
    })

# Filter to top 50 peaks per run using groupby
df_peaks = pd.DataFrame(peak_data)
df_peaks = df_peaks.groupby('run_no', group_keys=False).apply(lambda x: x.
    nlargest(50, 'peak_height'))

# 5. Generate Output
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# Summary Statistics
stats = {
    'Metric': ['Mean', 'Std', 'Max'],
    'UV_1_280_Residual': [np.mean(residuals_uv1), np.std(residuals_uv1), np.
        max(residuals_uv1)],
    'UV_2_260_Residual': [np.mean(residuals_uv2), np.std(residuals_uv2), np.
        max(residuals_uv2)]
}
stats_df = pd.DataFrame(stats)

# Baseline Stability Metrics (CV and Range on smoothed baseline values)
baseline_stability = {
    'UV_1_280_CV': np.std(smoothed_uv1) / (np.abs(np.mean(df_analysis['
        UV_1_280_mAU']))) + 1e-9),
    'UV_2_260_CV': np.std(smoothed_uv2) / (np.abs(np.mean(df_analysis['
        UV_2_260_mAU']))) + 1e-9),
    'UV_1_280_Range': np.max(smoothed_uv1) - np.min(smoothed_uv1),
    'UV_2_260_Range': np.max(smoothed_uv2) - np.min(smoothed_uv2)
}
stability_df = pd.DataFrame(baseline_stability, index=['CV', 'CV', 'Range', '
    Range'])
stability_df.columns = ['UV_1_280', 'UV_2_260']

```

```

# Combine all outputs into one text file
output_lines = []
output_lines.append("=" * 60)
output_lines.append("DATA_ANALYSIS_REPORT: BASELINE_CORRECTION & PEAK_
DETECTION")
output_lines.append("=" * 60)
output_lines.append("")

output_lines.append("1. SUMMARY STATISTICS OF ASLS RESIDUALS")
output_lines.append("-" * 40)
output_lines.append(stats_df.to_string(index=False))
output_lines.append("")

output_lines.append("2. DETECTED LOCAL MAXIMA (Top 50 per run)")
output_lines.append("-" * 40)
# Print all detected peaks (max 50 per run)
output_lines.append(df_peaks.to_string(index=False))
output_lines.append("")

output_lines.append("3. CONFIRMATION OF BASELINE STABILITY METRICS")
output_lines.append("-" * 40)
output_lines.append(stability_df.to_string(index=False))
output_lines.append("")
output_lines.append("=" * 60)
output_lines.append("END OF REPORT")
output_lines.append("=" * 60)

output_text = "\n".join(output_lines)

output_file_path = os.path.join(output_dir, 'baseline_correction_report.txt')
with open(output_file_path, 'w') as f:
    f.write(output_text)

print(f"Report saved to {output_file_path}")

# Execute the function
if __name__ == '__main__':
    # Data size check
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)
    df_filtered = df[(df['volume_ml'] >= 0) & (~pd.isna(df['Fraction_number']))]
    df_analysis = df_filtered[['run_no', 'Fraction_number', 'UV_1_280_mAU', '
UV_2_260_mAU']].copy()
    df_analysis = df_analysis.sort_values(['run_no', 'Fraction_number']).
        reset_index(drop=True)

    print(f"Input data shape: {df_analysis.shape}")
    print(f"Number of rows in df_analysis: {len(df_analysis)}")

    # Early exit on large data
    if len(df_analysis) > 10000:
        print("Warning: Dataset exceeds 10,000 rows. Skipping full processing to
        prevent timeout.")
        print("Consider chunking or reducing data size.")
        sys.exit(0)

call_code_updates()

```

Listing 1: Code.1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.ndimage import uniform_filter1d
import warnings
warnings.filterwarnings('ignore')

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Step 1: Normalization
    # Exclude rows where fraction_volume is missing AND (volume_ml is missing or
    # zero)
    mask_volume = df['volume_ml'].notna() & (df['volume_ml'] > 0)
    mask_fraction = df['fraction_volume'].notna()

    # Vectorized volume metric selection
    df['volume_metric'] = np.where(mask_fraction, df['fraction_volume'], np.where(
        mask_volume, df['volume_ml'], np.nan))

    # Exclude rows with missing volume metric
    df = df.dropna(subset=['volume_metric'])

    # Normalize traces
    df['UV_1_280_norm'] = df.groupby('run_no')['UV_1_280_mAU'].transform(lambda x:
        x / (x.max() + 1e-9))
    df['UV_2_260_norm'] = df.groupby('run_no')['UV_2_260_mAU'].transform(lambda x:
        x / (x.max() + 1e-9))

    # Step 2: Peak Extraction
    # Vectorized peak extraction on entire arrays
    peaks_280, _ = find_peaks(df['UV_1_280_norm'].values, prominence=0.05)
    peaks_260, _ = find_peaks(df['UV_2_260_norm'].values, prominence=0.05)

    df['Peak_280'] = peaks_280
    df['Peak_260'] = peaks_260

    # Merge peaks into a single dataframe
    peak_data = df[['run_no', 'Fraction_number', 'Peak_280', 'Peak_260']].copy()
    peak_data = peak_data.explode(['Peak_280', 'Peak_260'])

    # Step 3: Metric Calculation
    # Calculate W, W0.05, Rs, and Tailing Factor on the globally normalized trace
    global_trace_280 = df.groupby('run_no')['UV_1_280_norm'].transform('mean')
    global_trace_260 = df.groupby('run_no')['UV_2_260_norm'].transform('mean')

    # Vectorized W calculation
    def calc_w_vec(traces):
        result = np.full(len(traces), np.nan)
        for i, trace in enumerate(traces):
            if trace.max() <= 0:
                continue

```

```

        half_max = trace.max() / 2
        indices = np.where(trace >= half_max)[0]
        if len(indices) < 2:
            continue
        result[i] = indices[-1] - indices[0]
    return result

df['W_280'] = calc_w_vec(df['UV_1_280_norm'].values)
df['W_260'] = calc_w_vec(df['UV_2_260_norm'].values)

# Vectorized W0.05 calculation
def calc_w005_vec(traces):
    result = np.full(len(traces), np.nan)
    for i, trace in enumerate(traces):
        if trace.max() <= 0:
            continue
        threshold = trace.max() * 0.05
        indices = np.where(trace >= threshold)[0]
        if len(indices) < 2:
            continue
        result[i] = indices[-1] - indices[0]
    return result

df['W0_05_280'] = calc_w005_vec(df['UV_1_280_norm'].values)
df['W0_05_260'] = calc_w005_vec(df['UV_2_260_norm'].values)

# Vectorized Rs calculation
def calc_rs_vec(traces_280, traces_260):
    result = np.full(len(traces_280), np.nan)
    for i in range(len(traces_280)):
        if pd.isna(traces_280[i]) or pd.isna(traces_260[i]):
            continue
        result[i] = (traces_280[i].max() - traces_260[i].min()) / ((traces_280[i].max() + traces_260[i].min()) / 2)
    return result

df['Rs'] = calc_rs_vec(df['UV_1_280_norm'].values, df['UV_2_260_norm'].values)

# Vectorized Tailing Factor calculation
def calc_tailing_vec(traces):
    result = np.full(len(traces), np.nan)
    for i, trace in enumerate(traces):
        if trace.max() <= 0:
            continue
        peak_idx = np.argmax(trace)
        left_half = trace[:peak_idx]
        right_half = trace[peak_idx:]
        if len(left_half) < 2 or len(right_half) < 2:
            continue
        left_width = np.where(left_half >= trace[peak_idx] * 0.5)[0][-1] - np.
            where(left_half >= trace[peak_idx] * 0.5)[0][0]
        right_width = np.where(right_half >= trace[peak_idx] * 0.5)[0][-1] -
            np.where(right_half >= trace[peak_idx] * 0.5)[0][0]
        result[i] = (2 * (left_width + right_width)) / (left_width + 5 *
            right_width)
    return result

df['Tailing_Factor'] = calc_tailing_vec(df['UV_1_280_norm'].values)

```

```

# Step 4: Dynamic Thresholding
# Calculate IQR/MAD thresholds exclusively on the ~35% of records where #
# Conc_B# is present
df_with_conc_b = df[df['Conc_B'].notna()]
if len(df_with_conc_b) > 0:
    iqr_280 = df_with_conc_b['UV_1_280_norm'].quantile(0.75) - df_with_conc_b[
        'UV_1_280_norm'].quantile(0.25)
    mad_280 = np.median(np.abs(df_with_conc_b['UV_1_280_norm'] -
        df_with_conc_b['UV_1_280_norm'].median()))
    threshold_280 = iqr_280 + 3 * mad_280

    iqr_260 = df_with_conc_b['UV_2_260_norm'].quantile(0.75) - df_with_conc_b[
        'UV_2_260_norm'].quantile(0.25)
    mad_260 = np.median(np.abs(df_with_conc_b['UV_2_260_norm'] -
        df_with_conc_b['UV_2_260_norm'].median()))
    threshold_260 = iqr_260 + 3 * mad_260

    # Vectorized filtering
    df['Peak_280_filtered'] = np.where(df['Peak_280'] > threshold_280, df[
        'Peak_280'], np.nan)
    df['Peak_260_filtered'] = np.where(df['Peak_260'] > threshold_260, df[
        'Peak_260'], np.nan)
else:
    df['Peak_280_filtered'] = df['Peak_280']
    df['Peak_260_filtered'] = df['Peak_260']

# Step 5: Spectral Ratio
# Compute 280nm/260nm ratio for peak regions
df['Peak_280_260_Ratio'] = np.where((df['Peak_280_filtered'] > 0) & (df[
    'Peak_260_filtered'] > 0),
    df['UV_1_280_mAU'] / (df['UV_2_260_mAU']
        + 1e-9),
    np.nan)

# Prepare data for visualization
df['Fraction_number_binned'] = pd.cut(df['Fraction_number'], bins=10, labels=
    False).astype(int)

# Create scatter plot with hexbin
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Hexbin plot
hexbin_data = df[df['Peak_280_260_Ratio'].notna()].copy()
ax[0].hexbin(hexbin_data['Fraction_number_binned'], hexbin_data['
    Peak_280_260_Ratio'],
    C=np.where((hexbin_data['Rs'] >= 1.5) & (hexbin_data['
        Tailing_Factor'] <= 2.0), 1, 0),
    cmap='viridis', gridsize=50)
ax[0].set_xlabel('Fraction_number (binned)')
ax[0].set_ylabel('Peak_280_260_Ratio')
ax[0].set_title('Hexbin Plot (Color: Rs >= 1.5 & T <= 2.0)')
ax[0].grid(True)

# Scatter plot overlay
scatter_data = df[df['Peak_280_260_Ratio'].notna()].copy()
ax[0].scatter(scatter_data['Fraction_number_binned'], scatter_data['
    Peak_280_260_Ratio'],
    alpha=0.5, s=10, c='red')

```

```
# Histogram of 280/260 ratio
ax[1].hist(hexbin_data['Peak_280_260_Ratio'], bins='auto', edgecolor='black',
           alpha=0.7)
ax[1].set_xlabel('Peak_280_260_Ratio')
ax[1].set_ylabel('Frequency')
ax[1].set_title('Histogram of 280/260 Ratio Distribution')

plt.tight_layout()
plt.savefig('/workspace/peak_analysis_visualisation.png', dpi=300)
plt.close()
###
```

Listing 2: Code_2